

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO MÔN: IOT
Đề tài: Hệ thống quản lý IOT

Sinh viên thực hiện: Đỗ Hữu Việt

Mã sinh viên: B22DCPT304

Giảng viên hướng dẫn: TS. Nguyễn Quốc Uy

Hà Nội, 2026

MỤC LỤC

Chương 1: Giới thiệu.....	4
1. Mục đích dự án.....	4
2. Phạm vi ứng dụng.....	4
3. Công nghệ sử dụng.....	5
3.1. Phần cứng và vi điều khiển.....	5
3.2. Giao thức giao tiếp.....	5
3.3. Backend và cơ sở dữ liệu.....	6
3.4. Frontend.....	6
Chương 2: Thiết kế hệ thống.....	7
1. Phân tích yêu cầu.....	7
1.1. Yêu cầu chức năng.....	7
1.2. Yêu cầu phi chức năng.....	7
2. Tổng quan kiến trúc hệ thống.....	8
2.1. Mô hình tổng quan.....	8
2.2. Luồng hoạt động của hệ thống.....	8
3. Thiết kế phần cứng.....	9
4. Thiết kế giao diện người dùng.....	9
4.1. Giao diện Dashboard.....	9
4.2. Giao diện DataSensors.....	10
4.3. Giao diện Actions History.....	11
4.4. Giao diện Profile.....	12
5. Thiết kế luồng hoạt động.....	12
5.1. Luồng đọc giá trị cảm biến.....	12
5.2. Luồng điều khiển thiết bị.....	14
5.3. Luồng truy cập trang ActionsHistory.....	16
5.4. Luồng truy cập trang DataSensors.....	18
6. Thiết kế luồng hoạt động.....	20
7. Giao thức MQTT.....	21
7.1. Tổng quan về MQTT.....	21
7.2. Cấu hình MQTT trong hệ thống.....	21
Chương 3: Xây dựng hệ thống.....	22
1. Phần cứng (Hardware).....	22
1.1. ESP 32.....	22
1.2. DHT11.....	22
1.3. BH1750.....	23

1.4. LED.....	23
1.5. Code phần cứng.....	23
2. Backend.....	29
2.1. Tổng quan.....	29
2.2. Công nghệ sử dụng.....	29
2.3. Cấu trúc thư mục.....	29
2.4. Tích hợp MQTT.....	29
3. FrontEnd.....	30
3.1. Tổng quan.....	30
3.2. Công nghệ sử dụng.....	30
3.3. Cấu trúc thư mục.....	31
4. Tài liệu API.....	32
4.1. Endpoint lấy dữ liệu cảm biến.....	32
4.2. Endpoint lấy lịch sử điều khiển thiết bị.....	33
4.3. Endpoint lấy trạng thái thiết bị.....	34
4.4. Endpoint điều khiển thiết bị.....	34
Chương 4: Kết luận và đánh giá.....	35
4.1. Kết quả đạt được.....	35
4.2. Đánh giá ưu điểm.....	35
4.3. Đánh giá về nhược điểm.....	36
4.4. Kết luận.....	36

Chương 1: Giới thiệu

1. Mục đích dự án

Trong thời đại công nghệ hiện đại, việc giám sát và điều khiển các thiết bị từ xa đã trở thành nhu cầu thiết yếu trong nhiều lĩnh vực như nông nghiệp thông minh, nhà thông minh, và các hệ thống công nghiệp. Dự án "Hệ Thống Quản Lý IoT" được phát triển nhằm tạo ra một giải pháp toàn diện cho việc thu thập, lưu trữ và quản lý dữ liệu từ các thiết bị cảm biến IoT.

Mục tiêu chính của dự án là xây dựng một hệ thống IoT cho phép thu thập dữ liệu từ các thiết bị hoặc cảm biến, sau đó truyền dữ liệu lên hệ thống máy chủ để giám sát và điều khiển từ xa thông qua giao diện web.

Cụ thể, dự án hướng đến các mục tiêu sau:

- Kết nối các thiết bị IoT với hệ thống mạng để thu thập dữ liệu từ cảm biến hoặc thiết bị điều khiển.
- Truyền dữ liệu theo thời gian thực từ thiết bị IoT đến server thông qua giao thức mạng.
- Xây dựng hệ thống backend và API để xử lý, lưu trữ và quản lý dữ liệu từ các thiết bị.
- Xây dựng giao diện giám sát và điều khiển giúp người dùng theo dõi trạng thái thiết bị từ xa.
- Tự động hóa quá trình điều khiển thiết bị, giúp nâng cao hiệu quả quản lý và giảm sự phụ thuộc vào thao tác thủ công.

Thông qua dự án, hệ thống có thể minh họa cách một nền tảng IoT hoàn chỉnh hoạt động, bao gồm các thành phần: thiết bị – mạng truyền thông – máy chủ xử lý – giao diện người dùng.

2. Phạm vi ứng dụng

Hệ thống IoT trong dự án có thể được áp dụng rộng rãi trong nhiều lĩnh vực khác nhau nhờ khả năng kết nối và tự động hóa. Trong lĩnh vực nhà thông minh (Smart Home), hệ thống không chỉ cho phép điều khiển các thiết bị điện như đèn, quạt hay điều hòa từ xa mà còn có thể tự động điều chỉnh hoạt động của thiết bị dựa trên thói quen người dùng hoặc dữ liệu cảm biến.

Trong lĩnh vực giám sát môi trường, IoT đóng vai trò quan trọng trong việc thu thập và phân tích dữ liệu từ các cảm biến như nhiệt độ, độ ẩm, chất lượng không khí hoặc mức tiêu thụ năng lượng. Các cảm biến này hoạt động liên tục và gửi dữ liệu về hệ thống trung tâm để xử lý, từ đó có thể phát hiện sớm các vấn đề như ô nhiễm, rò rỉ nước hoặc biến đổi môi trường bất thường.

Hệ thống cũng có thể được mở rộng để phục vụ các ứng dụng nghiên cứu khoa học, giáo dục và phát triển sản phẩm. Khả năng lưu trữ và phân tích dữ liệu lịch sử tạo điều kiện cho việc nghiên cứu các xu hướng thay đổi môi trường theo thời gian.

3. Công nghệ sử dụng

Hệ thống được xây dựng dựa trên kiến trúc client-server với các thành phần công nghệ được lựa chọn phù hợp với từng lớp chức năng.

3.1. Phần cứng và vi điều khiển

Trong dự án, hệ thống phần cứng được xây dựng dựa trên vi điều khiển ESP32 kết hợp với các cảm biến DHT11 và BH1750 nhằm thu thập dữ liệu môi trường. ESP32 đóng vai trò là trung tâm xử lý chính, với khả năng xử lý mạnh mẽ nhờ kiến trúc vi xử lý 32-bit, tích hợp WiFi và Bluetooth, cho phép thiết bị dễ dàng kết nối Internet để truyền dữ liệu lên hệ thống server.

Cảm biến DHT11 được sử dụng để đo nhiệt độ và độ ẩm của môi trường. Đây là cảm biến kỹ thuật số tích hợp sẵn bộ chuyển đổi và vi điều khiển bên trong, cho phép xuất dữ liệu trực tiếp dưới dạng tín hiệu số với độ ổn định cao. DHT11 có khả năng đo nhiệt độ trong khoảng từ 0 đến 60°C và độ ẩm từ 20% đến 90%, phù hợp cho các ứng dụng giám sát môi trường cơ bản như nhà thông minh hoặc hệ thống theo dõi điều kiện phòng.

Bên cạnh đó, cảm biến BH1750 được sử dụng để đo cường độ ánh sáng môi trường. Đây là cảm biến ánh sáng kỹ thuật số có độ phân giải cao, giao tiếp với ESP32 thông qua giao thức I2C, giúp việc kết nối trở nên đơn giản và ổn định. BH1750 có thể đo độ sáng trong dải rộng và trả về giá trị dưới đơn vị lux, từ đó hệ thống có thể xác định điều kiện ánh sáng để tự động điều chỉnh thiết bị như đèn chiếu sáng hoặc các hệ thống liên quan.

3.2. Giao thức giao tiếp

Giao thức MQTT (Message Queuing Telemetry Transport) được sử dụng để truyền dữ liệu giữa thiết bị IoT và server. Đây là một giao thức truyền thông nhẹ, được thiết kế đặc biệt cho các hệ thống IoT có tài nguyên hạn chế và mạng

không ổn định. MQTT hoạt động dựa trên mô hình publish/subscribe, trong đó các thiết bị (publisher) gửi dữ liệu lên một máy chủ trung gian gọi là broker, và các ứng dụng hoặc hệ thống khác (subscriber) sẽ nhận dữ liệu thông qua các topic đã đăng ký

MQTT giúp giảm sự phụ thuộc trực tiếp giữa các thiết bị, tăng tính linh hoạt và khả năng mở rộng của hệ thống. Giao thức này có ưu điểm là tiêu tốn ít băng thông, yêu cầu tài nguyên thấp và phù hợp với các thiết bị như ESP32

3.3. Backend và cơ sở dữ liệu

Ở Backend, hệ thống sử dụng Nodejs kết hợp với thư viện Express để xây dựng server và các API. Nodejs là môi trường chạy JavaScript phía server, cho phép xử lý các yêu cầu mạng hiệu quả, trong khi Express là framework giúp đơn giản hóa việc xây dựng API, định tuyến (routing) và xử lý request/response trong ứng dụng web. Backend có nhiệm vụ nhận dữ liệu từ thiết bị IoT thông qua MQTT, xử lý logic và cung cấp dữ liệu cho frontend.

Về cơ sở dữ liệu, hệ thống sử dụng MySQL, là một hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) phổ biến. MySQL được sử dụng để lưu trữ dữ liệu cảm biến như nhiệt độ, độ ẩm và ánh sáng, cũng như lịch sử hoạt động của hệ thống. Với cấu trúc dữ liệu dạng bảng rõ ràng và khả năng truy vấn bằng SQL, MySQL giúp đảm bảo tính nhất quán, dễ quản lý và phù hợp với các ứng dụng cần lưu trữ dữ liệu có cấu trúc ổn định

3.4. Frontend

Hệ thống được xây dựng bằng các công nghệ cơ bản gồm HTML, CSS và JavaScript. HTML dùng để xây dựng cấu trúc giao diện, CSS dùng để thiết kế và định dạng hiển thị, trong khi JavaScript đảm nhiệm việc xử lý tương tác và gọi API từ backend để hiển thị dữ liệu lên giao diện. Các file tĩnh như HTML, CSS và JavaScript có thể được phục vụ trực tiếp từ server Express, giúp tạo ra một giao diện web trực quan cho phép người dùng theo dõi dữ liệu và điều khiển hệ thống IoT một cách dễ dàng

Chương 2: Thiết kế hệ thống

1. Phân tích yêu cầu

1.1. Yêu cầu chức năng

Yêu cầu chức năng của hệ thống IoT tập trung vào các chức năng chính mà hệ thống phải cung cấp cho người dùng và thiết bị. Trước hết, hệ thống phải có khả năng thu thập dữ liệu từ các cảm biến như nhiệt độ, độ ẩm và ánh sáng thông qua vi điều khiển ESP32. Dữ liệu cảm biến cần được thu thập liên tục với tần suất 2 giây 1 lần và truyền tải qua giao thức MQTT đến backend server.

Hệ thống cần hỗ trợ lưu trữ dữ liệu vào cơ sở dữ liệu MySQL, giúp lưu lại lịch sử và phục vụ cho việc truy vấn sau này. Người dùng có thể xem dữ liệu cảm biến trên giao diện web dưới dạng số liệu hoặc biểu đồ trực quan. Bên cạnh đó, hệ thống cũng phải cho phép điều khiển thiết bị từ xa (ví dụ bật/tắt đèn) thông qua giao diện frontend, và các lệnh này sẽ được gửi từ backend xuống thiết bị thông qua MQTT.

Hệ thống cũng cần đảm bảo các chức năng như cập nhật dữ liệu theo thời gian thực, quản lý trạng thái thiết bị và giao tiếp hai chiều giữa thiết bị và server.

1.2. Yêu cầu phi chức năng

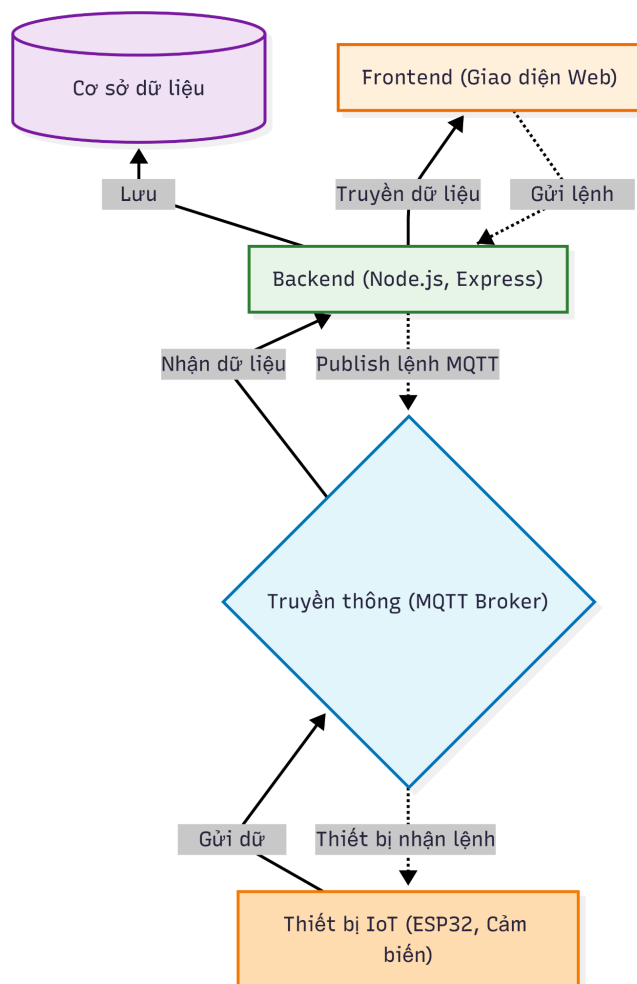
Bên cạnh các chức năng chính, hệ thống cũng cần đáp ứng các yêu cầu phi chức năng nhằm đảm bảo chất lượng và hiệu năng hoạt động. Trước hết, hệ thống cần đảm bảo hiệu năng (performance), cụ thể là dữ liệu từ cảm biến phải được cập nhật nhanh chóng, gần như theo thời gian thực, và giao diện web phải phản hồi nhanh khi người dùng thao tác.

Về độ tin cậy (reliability), hệ thống phải hoạt động ổn định trong thời gian dài, đảm bảo dữ liệu không bị mất trong quá trình truyền và lưu trữ. Ngoài ra, hệ thống cần có khả năng mở rộng (scalability) để có thể kết nối thêm nhiều thiết bị hoặc cảm biến trong tương lai mà không ảnh hưởng đến hiệu suất.

Về bảo mật (security), hệ thống cần đảm bảo chỉ những người dùng hợp lệ mới có thể truy cập và điều khiển thiết bị, đồng thời dữ liệu truyền qua mạng cần được bảo vệ khỏi các truy cập trái phép. Bên cạnh đó, yếu tố khả dụng (usability) cũng rất quan trọng, giao diện cần thân thiện, dễ sử dụng và hiển thị thông tin rõ ràng cho người dùng.

2. Tổng quan kiến trúc hệ thống

2.1. Mô hình tổng quan



2.2. Luồng hoạt động của hệ thống

Luồng dữ liệu trong hệ thống được thiết kế theo hướng một chiều từ thiết bị cảm biến đến người dùng cuối:

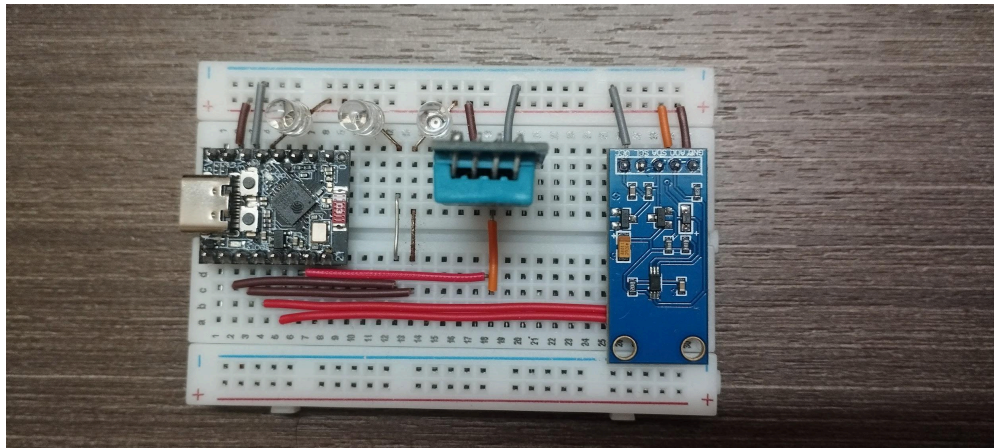
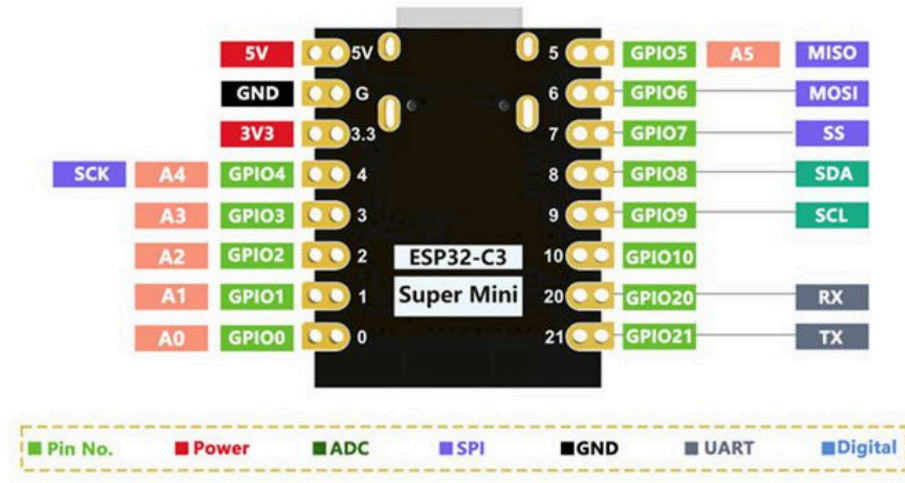
Bước 1: Thu thập dữ liệu: ESP32 đọc dữ liệu từ cảm biến mỗi giây và đóng gói thành định dạng JSON

Bước 2: Truyền tải: Dữ liệu được gửi qua MQTT broker đến backend API

Bước 3: Xử lý: Backend nhận dữ liệu, kiểm tra tính hợp lệ, tính toán trạng thái và lưu trữ vào mySql

Bước 4: Hiển thị: Frontend gọi API để lấy dữ liệu và cập nhật giao diện thời gian thực

3. Thiết kế phần cứng



Các linh kiện được kết nối với ESP trên các chân

- DHT Data Pin: GPIO10
- BH1750 SDA Pin: GPIO8
- BH1750 SCL Pin: GPIO9
- LED1 Pin: GPIO4
- LED2 Pin: GPIO6
- LED3 Pin: GPIO7

4. Thiết kế giao diện người dùng

4.1. Giao diện Dashboard

Dashboard cung cấp cái nhìn tổng quan về trạng thái cảm biến và các chức năng điều khiển cơ bản.

Mỗi loại cảm biến được hiển thị trong card riêng biệt với:

- Tên cảm biến và đơn vị đo

- Giá trị hiện tại được cập nhật thời gian thực
- Hệ thống điều khiển LED được tích hợp trực tiếp trên trang chủ với:
 - Toggle switch cho mỗi LED (LED1, LED2, LED3)
 - Trạng thái hiện tại (BẬT/TẮT)

Biểu đồ sử dụng Chart.js để hiển thị xu hướng dữ liệu cảm biến trong khoảng thời gian ngắn hoặc hiển thị toàn bộ dữ liệu, giúp người dùng theo dõi thay đổi của thông số môi trường.



4.2. Giao diện DataSensors

Trang dữ liệu cảm biến được thiết kế để quản lý và phân tích dữ liệu lịch sử gồm:

- Bảng dữ liệu có thể sắp xếp theo các cột khác nhau
- Hệ thống phân trang
- Tìm kiếm theo thời gian và lọc theo cảm biến (nhiệt độ, độ ẩm, ánh sáng)

ID	Cảm biến	Giá trị cảm biến	Thời gian
#1001	Độ ẩm	60 %	2026-01-20 08:31:11
#1002	Độ ẩm	60 %	2026-01-20 08:31:12
#1003	Độ ẩm	60 %	2026-01-20 08:31:13
#1004	Ánh sáng	300 lx	2026-01-20 08:31:14
#1005	Nhiệt độ	25.5 °C	2026-01-20 08:31:15
#1006	Độ ẩm	60 %	2026-01-20 08:31:16
#1007	Ánh sáng	300 lx	2026-01-20 08:31:17
#1008	Độ ẩm	60 %	2026-01-20 08:31:18
#1009	Ánh sáng	300 lx	2026-01-20 08:31:19
#1010	Nhiệt độ	25.5 °C	2026-01-20 08:31:20

4.3. Giao diện Actions History

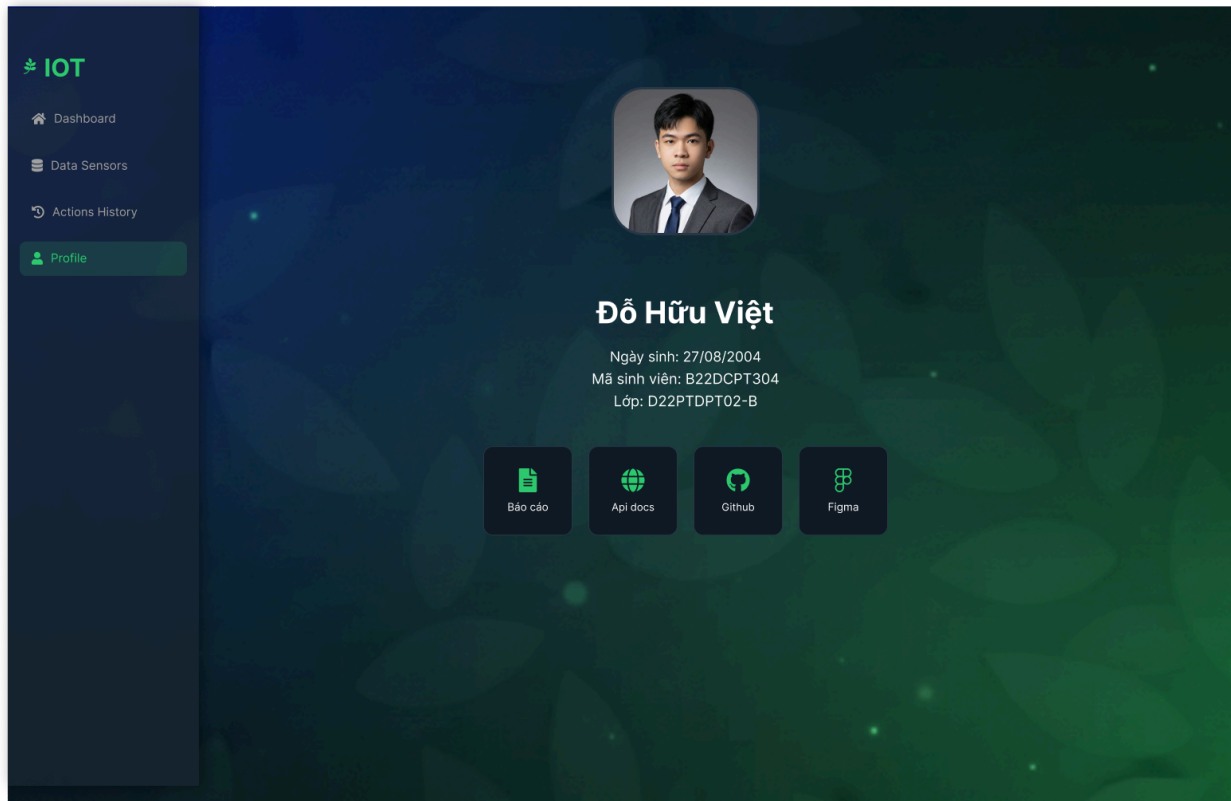
Trang lịch sử hoạt động theo dõi tất cả các thao tác điều khiển LED gồm:

- Bảng lịch sử hoạt động
- + Hiển thị chi tiết từng hành động với timestamp
- + Thông tin thiết bị (LED1, LED2, LED3)
- Lọc theo thiết bị cụ thể
- Tìm kiếm theo thời gian

ID	Thiết bị	Hành động	Trạng thái	Thời gian
1	Đèn	ON	Thành công	2026-01-20 20:30:15
2	Quạt	OFF	Đang xử lý	2026-01-20 20:35:20
3	Điều hòa	ON	Thất bại	2026-01-20 20:40:05
4	Điều hòa	OFF	Đang xử lý	2026-01-21 08:20:00
5	Đèn	OFF	Thành công	2026-01-21 08:45:10
6	Quạt	OFF	Thành công	2026-01-21 09:15:20
7	Điều hòa	ON	Thất bại	2026-01-21 09:30:05
8	Đèn	ON	Thành công	2026-01-21 09:55:30
9	Quạt	ON	Thành công	2026-01-21 10:10:12
10	Điều hòa	OFF	Thành công	2026-01-21 10:15:45

4.4. Giao diện Profile

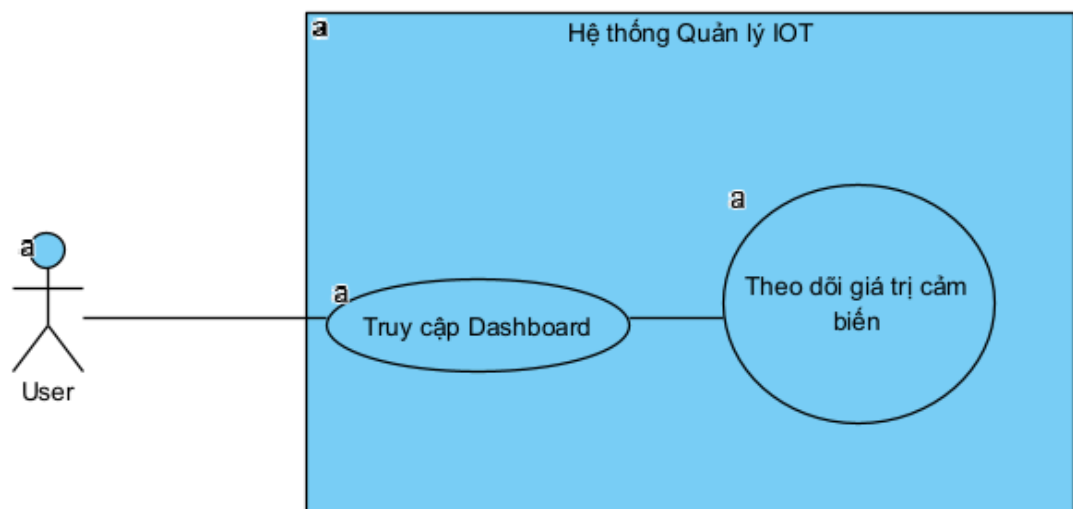
Trang profile cung cấp giao diện hiển thị thông tin cá nhân người dùng



5. Thiết kế luồng hoạt động

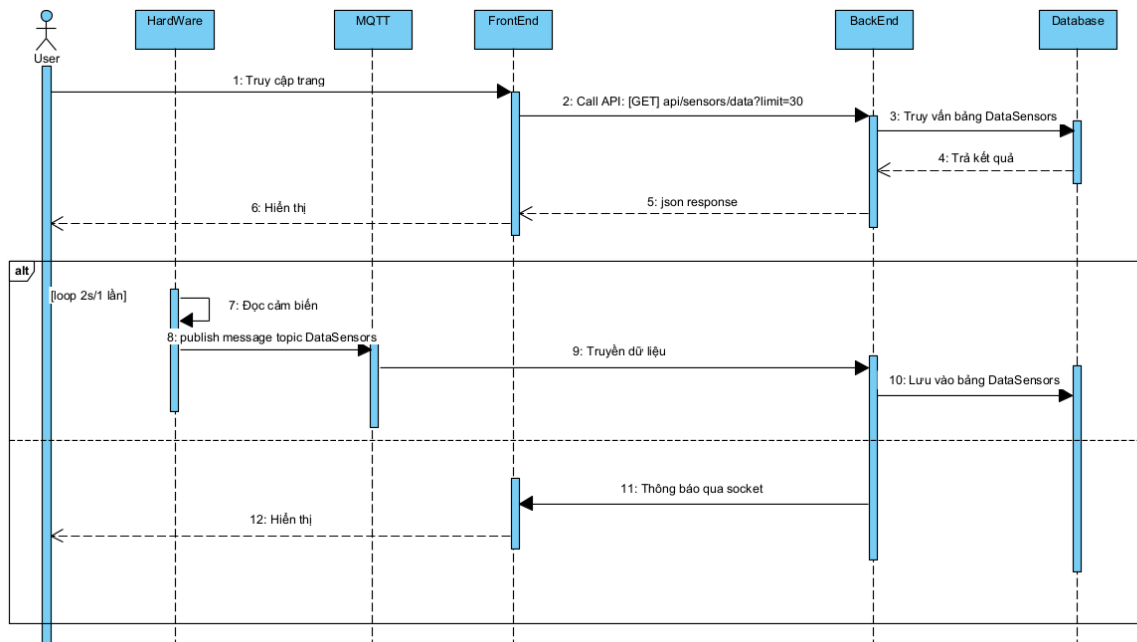
5.1. Luồng đọc giá trị cảm biến

- Use case:



- Truy cập Dashboard: Cho phép người dùng truy nhập trang Dashboard
- Theo dõi giá trị cảm biến: UC này cho phép người dùng theo dõi giá trị cảm biến realtime

● Sequence:



Giai đoạn 1: Truy cập và hiển thị dữ liệu ban đầu

1. User thực hiện truy nhập vào trang dashboard
2. FrontEnd gửi một yêu cầu GET `api/sensors/data?limit = 30` đến BackEnd để lấy 30 dữ liệu mới nhất sử dụng để vẽ biểu đồ.
3. BackEnd truy vấn đến bảng DataSensors trong Database.
4. Database trả kết quả truy vấn về cho BackEnd.
5. BackEnd đóng gói dữ liệu và gửi phản hồi dạng JSON response về cho FrontEnd.
6. FrontEnd xử lý dữ liệu và hiển thị lên giao diện cho User

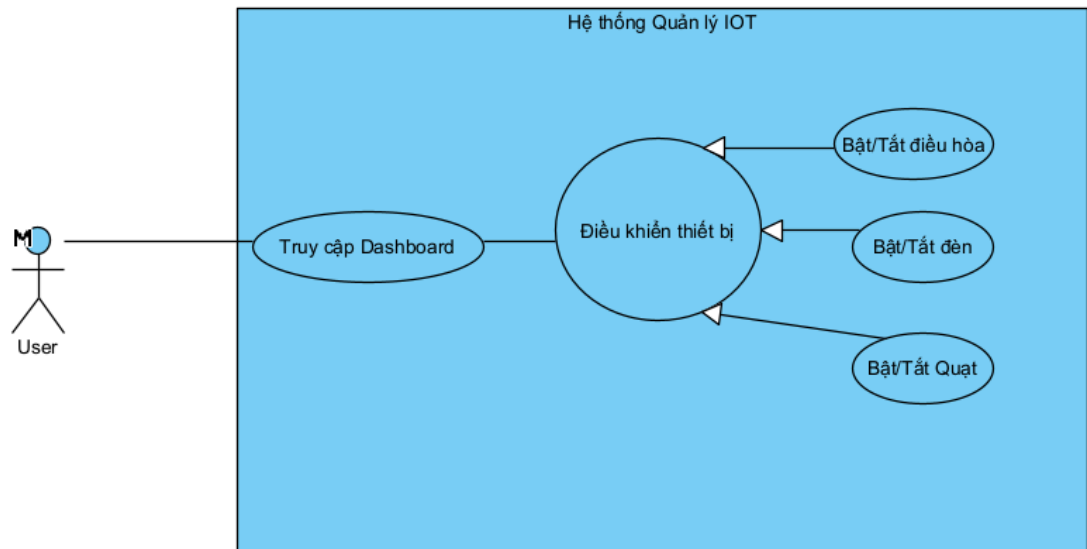
Giai đoạn 2: Vòng lặp cập nhật dữ liệu thời gian thực (Loop 2 giây/lần)

7. HardWare thực hiện đọc dữ liệu từ các cảm biến.
8. HardWare gửi (publish) dữ liệu này dưới dạng một message lên một topic có tên là DataSensors trên MQTT.
9. MQTT truyền dữ liệu nhận được tới BackEnd.
10. BackEnd nhận dữ liệu và thực hiện lưu trữ vào bảng DataSensors trong Database.

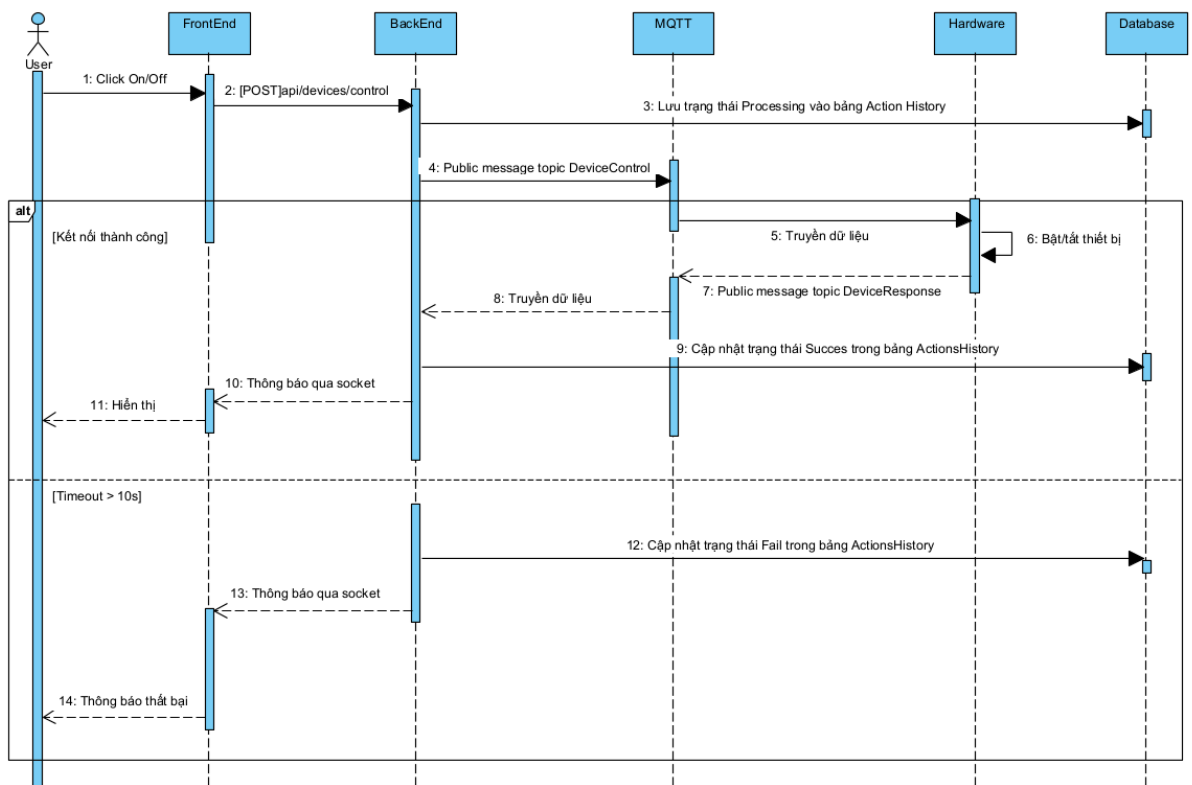
- 11.BackEnd gửi một thông báo qua kết nối Socket tới FrontEnd để báo rằng có dữ liệu mới.
- 12.FrontEnd nhận được tín hiệu từ Socket, ngay lập tức cập nhật và hiển thị giá trị mới nhất lên màn hình cho User.

5.2. Luồng điều khiển thiết bị

- Use case:



- Truy cập Dashboard: Cho phép người dùng truy nhập trang Dashboard
 - Điều khiển thiết bị: Cho phép người dùng điều khiển các thiết bị thông qua web
 - Bật/Tắt điều hòa: UC này cho phép người dùng bật/tắt điều hòa
 - Bật/Tắt đèn: UC này cho phép người dùng bật/tắt đèn
 - Bật/Tắt quạt: UC này cho phép người dùng bật/tắt quạt
- Sequence:



1. User thực hiện nhấn nút Click On/Off trên giao diện người dùng (FrontEnd).
2. FrontEnd gọi API POST api/devices/control gửi yêu cầu đến BackEnd.
3. BackEnd thực hiện ghi nhận yêu cầu lưu lịch sử hành động bằng cách thêm dữ liệu vào bảng Action History trong Database với trạng thái Processing.
4. BackEnd đồng thời gửi public message đến MQTT với topic DeviceControl.
5. MQTT thực hiện truyền dữ liệu đến HardWare.
6. Hardware nhận lệnh và thực hiện hành động: Bật/tắt thiết bị tương ứng.

Nhánh 1: Phản hồi thành công

7. Sau khi thực hiện, Hardware public ngược lại lên topic DataResponse của MQTT.
8. MQTT tiếp tục truyền dữ liệu phản hồi này về cho BackEnd.
9. BackEnd nhận phản hồi và tiến hành cập nhật trạng thái Success của thiết bị trong bảng ActionsHistory ở Database.
10. BackEnd gửi thông báo socket kết quả thành công về cho FrontEnd.

11. FrontEnd xử lý phản hồi và hiển thị trạng thái mới nhất của thiết bị lên màn hình cho User.

Nhánh 2: Chờ phản hồi quá 10 giây

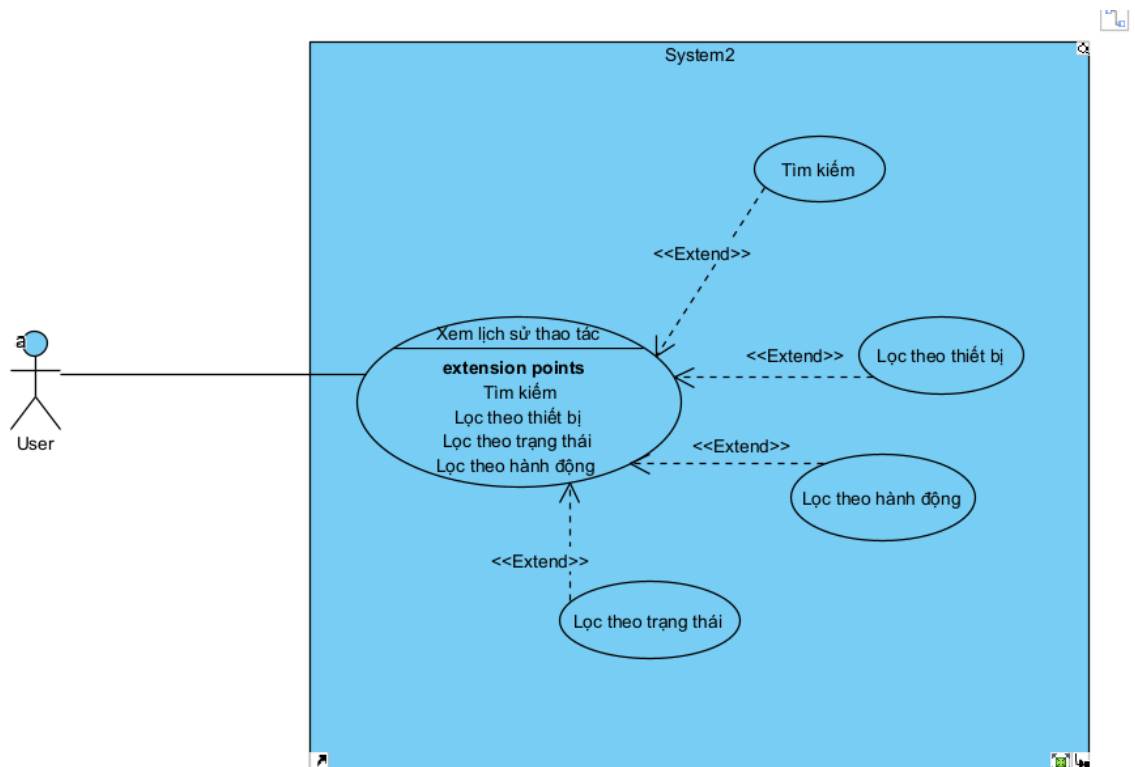
12. Backend cập nhật trạng thái Fail của thiết bị trong bảng ActionsHistory

13. Backend gửi thông báo socket thất bại về cho FrontEnd

14. FrontEnd hiển thị thông báo cho người dùng

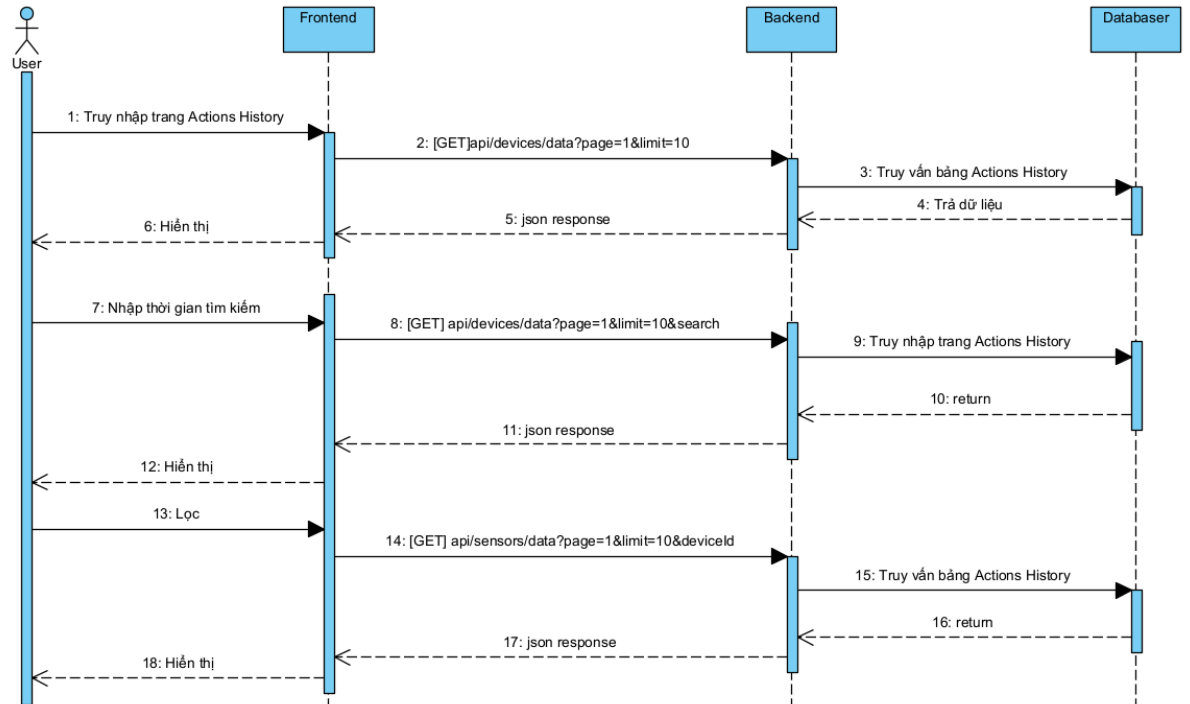
5.3. Luồng truy cập trang ActionsHistory

- Use case:



- Xem lịch sử thao tác: Use case này cho phép người dùng xem lịch sử thao tác với thiết bị
- Tìm kiếm: Use case này cho phép người dùng tìm kiếm theo thời gian
- Lọc theo thiết bị: Use case này cho phép người dùng lọc bảng lịch sử theo thiết bị
- Lọc theo hành động: Use case này cho phép người dùng lọc theo hành động
- Lọc theo trạng thái: Use case này cho phép người dùng lọc theo trạng thái của thiết bị

- Sequence:



1. User thực hiện truy nhập trang Actions History.
2. Frontend gọi API lấy lịch sử với tham số phân trang (page=1&limit=10) gửi đến Backend.
3. Backend thực hiện truy vấn bảng Actions History trong Database.
4. Database thực hiện trả dữ liệu về cho Backend.
5. Backend gửi response chứa danh sách lịch sử về Frontend.
6. Frontend hiển thị danh sách cho User.

Nếu User thực hiện tìm kiếm:

7. User thực hiện Nhập nội dung tìm kiếm
8. Frontend gửi yêu cầu API kèm từ khóa tìm kiếm (&search) đến Backend.
9. Backend thực hiện truy vấn bảng Actions History theo yêu cầu mới.
10. Database trả về kết quả
11. Backend gửi response kết quả tìm kiếm về cho Frontend.
12. Frontend cập nhật và hiển thị kết quả tìm kiếm cho User.

Nếu User thực hiện lọc:

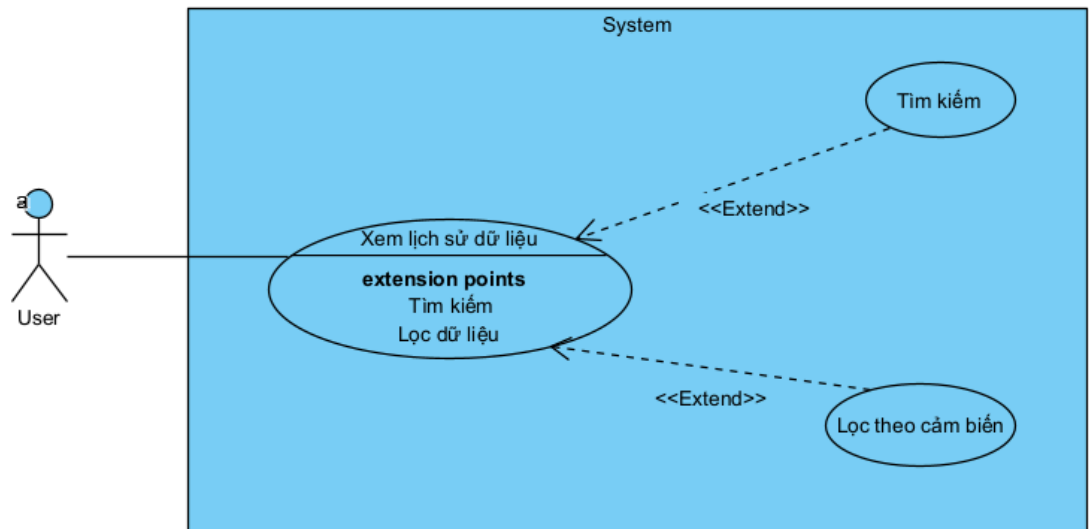
13. User chọn lọc theo cảm biến
14. Frontend gửi yêu cầu API kèm (&deviceId) đến Backend.
15. Backend thực hiện truy vấn bảng Actions History theo yêu cầu mới.
16. Database trả về kết quả

17.Backend gửi response kết quả lọc về cho Frontend.

18.Frontend cập nhật và hiển thị kết quả tìm kiếm cho User.

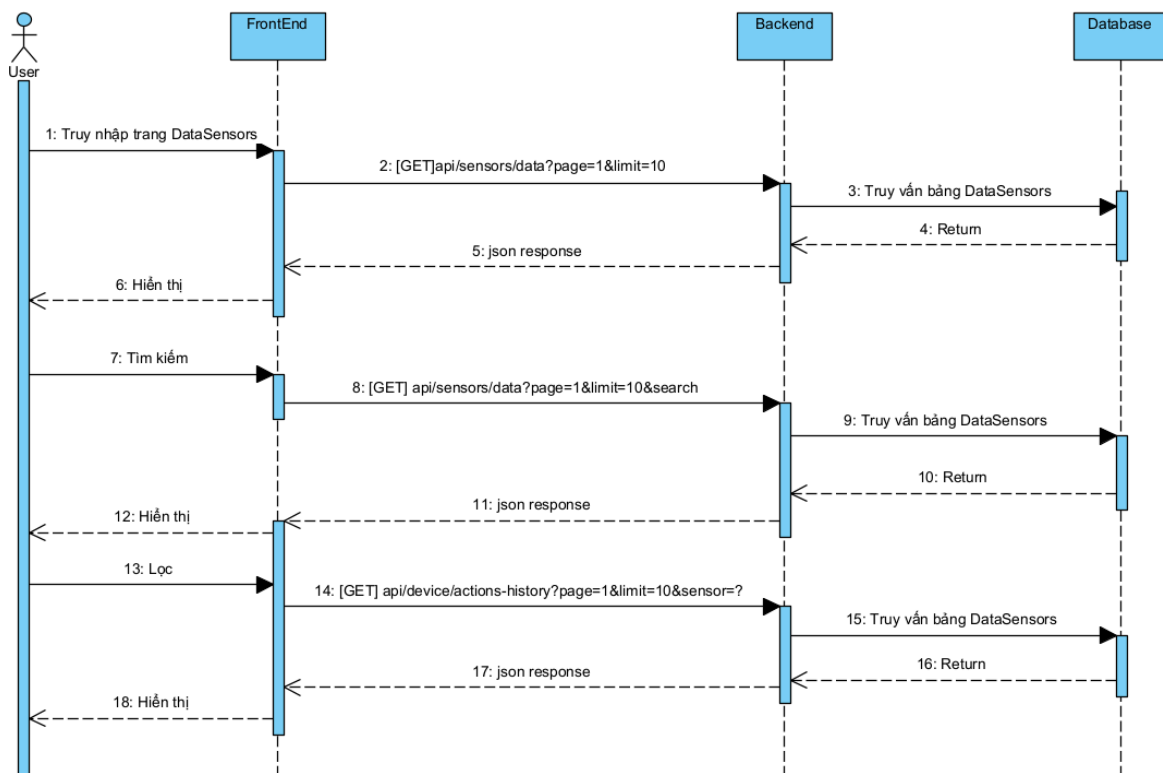
5.4. Luồng truy cập trang DataSensors

- Use case:



- Xem lịch sử dữ liệu: Use case này cho phép người dùng xem lịch sử thu thập dữ liệu cảm biến
- Tìm kiếm: Use case này cho phép người dùng tìm kiếm theo thời gian
- Lọc theo cảm biến: Use case này cho phép lọc theo cảm biến

- Sequence:



1. User thực hiện hành động truy cập trang DataSensors.
2. FrontEnd gửi yêu cầu GET api/sensors/data lấy danh sách dữ liệu (có kèm tham số phân trang page=1&limit=10) đến Backend.
3. Backend thực hiện Truy vấn bảng DataSensors trong Database.
4. Database trả về tập dữ liệu tương ứng.
5. Backend gửi phản hồi (response) về cho FrontEnd.
6. FrontEnd xử lý và Hiển thị danh sách dữ liệu lên màn hình cho User.

Nếu User tiến hành tìm kiếm:

7. User nhập từ khóa và nhấn Tìm kiếm.
8. FrontEnd gửi yêu cầu GET api/sensors/sensors-data-list?page=1&limit=10 với tham số tìm kiếm (&search) đến Backend.
9. Backend thực hiện truy vấn bảng DataSensors dựa trên điều kiện lọc/tìm kiếm.
10. Database trả về kết quả đã lọc.
11. Backend gửi phản hồi (response) về cho FrontEnd.
12. FrontEnd cập nhật giao diện và hiển thị kết quả tìm kiếm cho User.

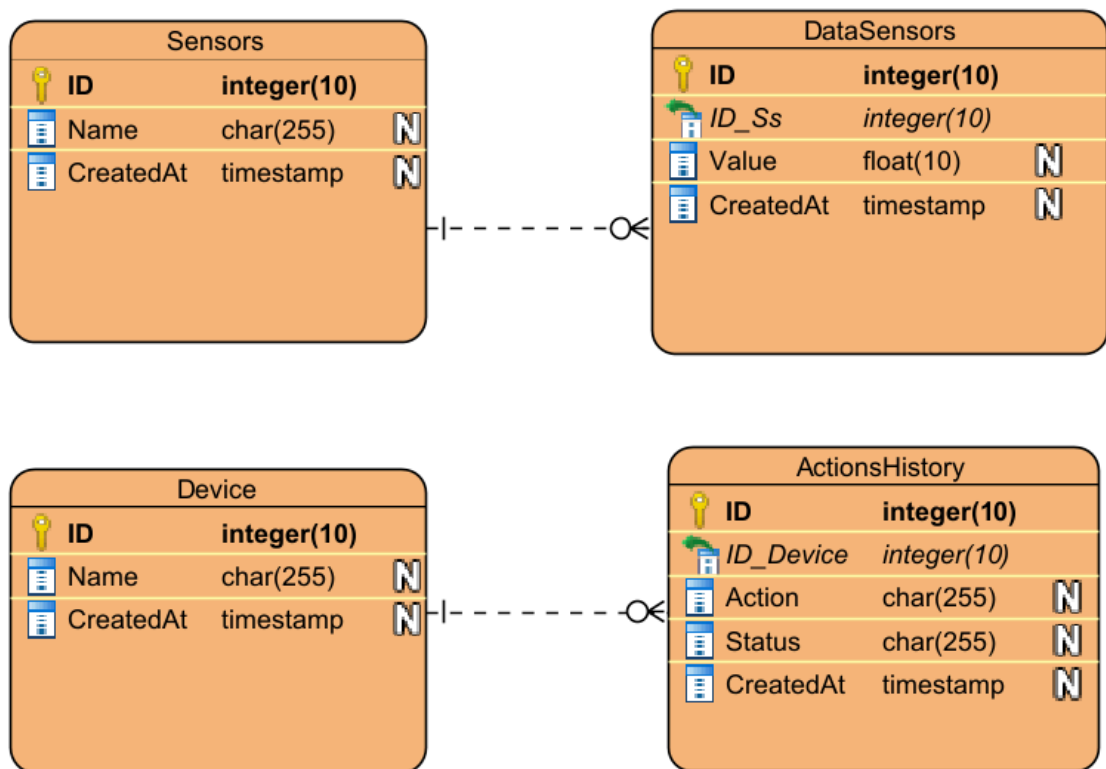
Nếu User tiến hành lọc:

13. User chọn tiêu chí lọc.

14. FrontEnd gửi yêu cầu GET
api/sensors/sensors-data-list?page=1&limit=10 với tham số (&sensor) đến Backend.
15. Backend thực hiện truy vấn bảng DataSensors dựa trên điều kiện lọc/tìm kiếm.
16. Database trả về kết quả đã lọc.
17. Backend gửi phản hồi (response) về cho FrontEnd.
18. FrontEnd cập nhật giao diện và hiển thị kết quả tìm kiếm cho User

6. Thiết kế luồng hoạt động

- Gồm 4 bảng: Sensors, DataSensors, Device, ActionsHistory



- Bảng Sensors lưu thông tin các cảm biến có các thuộc tính gồm ID kiểu int, Name kiểu varchar, CreatedAt kiểu timestamp. Bảng Sensors có ID là khóa chính.
- Lớp Sensors và lớp DataSensors có quan hệ thành phần 1-n nên bảng DataSensors chứa ID_Ss.
- Bảng DataSensors lưu thông tin dữ liệu của cảm biến có các thuộc tính gồm ID kiểu int, IDSs kiểu int, Value kiểu float, CreatedAt kiểu timestamp. Bảng DataSensors có ID là khóa chính, ID_Ss là khóa ngoại.

- Bảng Device lưu thông tin các thiết bị có các thuộc tính kiểu ID kiểu int, Name kiểu varchar, CreatedAt kiểu timestamp. Bảng Device có ID là khóa chính
- Lớp Device và lớp ActionsHistory có quan hệ thành phần 1-n nên bảng ActionsHistory chứa ID_Device
- Bảng ActionsHistory lưu thông tin lịch sử hoạt động của các thiết bị gồm ID kiểu int, ID_Device kiểu int, Action kiểu varchar, Status kiểu varchar và CreatedAt kiểu timestamp

7. Giao thức MQTT

7.1. Tổng quan về MQTT

MQTT (Message Queuing Telemetry Transport) là giao thức truyền thông publish-subscribe nhẹ, được thiết kế đặc biệt cho các ứng dụng IoT

Kiến trúc MQTT:

- Broker: Server trung tâm quản lý connections và message routing
- Client: Thiết bị publish hoặc subscribe messages
- Topic: Địa chỉ logic để phân loại messages
- Message: Payload dữ liệu được truyền tải

7.2. Cấu hình MQTT trong hệ thống

- Cài đặt kết nối:
- Host: localhost
- Port: 8888 (TLS/SSL)
- Protocol: MQTT over TLS
- Authentication: Username/Password authentication

Chương 3: Xây dựng hệ thống

1. Phần cứng (Hardware)

Hệ thống phần cứng của dự án được xây dựng theo mô hình IoT cơ bản, trong đó ESP32 đóng vai trò là bộ điều khiển trung tâm, có nhiệm vụ kết nối mạng WiFi, đọc dữ liệu từ cảm biến và điều khiển các thiết bị đầu ra. Hệ thống sử dụng hai cảm biến chính là DHT11 để đo nhiệt độ và độ ẩm, cùng với BH1750 để đo cường độ ánh sáng. Bên cạnh đó, hệ thống còn sử dụng ba đèn LED đại diện cho các thiết bị có thể được điều khiển từ xa thông qua MQTT. Toàn bộ phần cứng được kết nối và vận hành theo chu trình:

Cảm biến thu thập dữ liệu → ESP32 xử lý → gửi dữ liệu lên server qua MQTT → nhận lệnh điều khiển từ server → xuất tín hiệu điều khiển ra thiết bị.

1.1. ESP 32

ESP32 là thành phần quan trọng nhất của hệ thống, đóng vai trò như “bộ não” điều khiển toàn bộ quá trình hoạt động. ESP32 được sử dụng để:

- Kết nối WiFi với mạng nội bộ.
- Giao tiếp với broker MQTT để gửi và nhận dữ liệu.
- Đọc dữ liệu từ cảm biến DHT11 và BH1750.
- Điều khiển các chân GPIO để bật/tắt LED theo lệnh nhận được.

ESP32 được lựa chọn vì có khả năng xử lý tốt, tích hợp sẵn WiFi, hỗ trợ giao tiếp với nhiều loại cảm biến và phù hợp với mô hình IoT cần truyền dữ liệu theo thời gian thực. Trong hệ thống này, ESP32 vừa làm nhiệm vụ thu thập dữ liệu, vừa làm nhiệm vụ truyền thông và điều khiển, giúp giảm số lượng linh kiện trung gian và đơn giản hóa kiến trúc phần cứng.

1.2. DHT11

Cảm biến DHT11 được kết nối với ESP32 để thu thập dữ liệu môi trường định kỳ. Dữ liệu đo được sau đó sẽ được ESP32 đọc bằng thư viện DHT.h, kiểm tra tính hợp lệ rồi gửi lên server qua MQTT.

Vai trò của DHT11 trong hệ thống là cung cấp thông tin về trạng thái môi trường xung quanh, phục vụ cho các ứng dụng như giám sát phòng học, nhà thông minh hoặc hệ thống theo dõi môi trường cơ bản. Đây là cảm biến có cấu

tạo đơn giản, dễ sử dụng, chi phí thấp và phù hợp với các dự án học tập, nghiên cứu.

1.3. BH1750

BH1750 là cảm biến dùng để đo cường độ ánh sáng, đơn vị là lux. Trong code, cảm biến này được kết nối với ESP32 qua giao tiếp I2C.

BH1750 có ưu điểm là giao tiếp đơn giản, độ chính xác tốt và không cần hiệu chuẩn phức tạp. Trong hệ thống, cảm biến này giúp theo dõi mức độ sáng của môi trường để có thể ứng dụng trong các bài toán như tự động bật đèn khi trời tối, giám sát ánh sáng trong phòng hoặc điều khiển thiết bị theo điều kiện ánh sáng thực tế.

Việc sử dụng giao tiếp I2C giúp ESP32 chỉ cần hai đường tín hiệu SDA và SCL để đọc dữ liệu từ cảm biến, nhờ đó tiết kiệm chân GPIO và giúp mạch gọn hơn.

1.4. LED

Trong hệ thống có ba đèn LED được khai báo tại các chân GPIO 4,6,7

Ba LED này đóng vai trò như các thiết bị đầu ra, mô phỏng cho các thiết bị điện có thể điều khiển từ xa trong hệ thống IoT. Khi ESP32 nhận được lệnh điều khiển từ MQTT

Việc sử dụng LED giúp mô phỏng trực quan trạng thái thiết bị, dễ quan sát khi hệ thống hoạt động. Trong các dự án thực tế, các LED này có thể được thay bằng relay để điều khiển đèn điện, quạt hoặc các thiết bị điện dân dụng khác

1.5. Code phần cứng

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <Wire.h>
#include <DHT.h>
#include <BH1750.h>

// ===== WIFI =====
const char* ssid = "Redmi Note 11";
const char* password = "12345679";

// ===== MQTT =====
const char* mqtt_server = "10.72.10.90";
```

```

const int mqtt_port = 1888;
const char* mqtt_user = "DoHuuViet";
const char* mqtt_pass = "viet123456";

const char* topic_control = "DeviceControl";
const char* topic_response = "DeviceResponse";
const char* topic_sensor = "DataSensors";
const char* topic_sync = "DeviceSync";
const char* topic_sync_request = "DeviceSyncRequest";

WiFiClient espClient;
PubSubClient client(espClient);

// ===== LED =====
#define LED1 4
#define LED2 6
#define LED3 7

// ===== DHT =====
#define DHTPIN 10
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

// ===== BH1750 =====
#define SDA_PIN 8
#define SCL_PIN 9
BH1750 lightMeter;
// ===== AUTO CONTROL =====
bool led1State = false;
// TIMER
unsigned long lastSend = 0;
const long interval = 2000;

// WIFI
void setup_wifi() {
  WiFi.begin(ssid, password);
  Serial.println("Connecting WiFi...");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
  }
  Serial.println("WiFi Connected");
}

```



```

//APPLY STATE
void applyState(int deviceID, String action) {
    int pin = -1;

    if (deviceID == 1) pin = LED1;
    else if (deviceID == 2) pin = LED2;
    else if (deviceID == 3) pin = LED3;

    if (pin != -1) {
        digitalWrite(pin, action == "ON" ? HIGH : LOW);
        if (deviceID == 1) {
            led1State = (action == "ON");
        }
    }
}

//SEND SYNC REQUEST
void requestSync() {
    StaticJsonDocument<100> doc;
    doc["request"] = "sync";

    String payload;
    serializeJson(doc, payload);

    client.publish(topic_sync_request, payload.c_str());
}

// CALLBACK
void callback(char* topic, byte* payload, unsigned int length) {

    String message = "";
    for (int i = 0; i < length; i++) {
        message += (char)payload[i];
    }

    StaticJsonDocument<200> doc;
    if (deserializeJson(doc, message)) return;

    if (String(topic) == topic_control) {
        int deviceID = doc["DeviceID"];
        String action = doc["Action"];

        applyState(deviceID, action);
    }
}

```

```

    StaticJsonDocument<100> res;
    res["DeviceID"] = deviceID;
    res["Status"] = "Success";
    res["Action"] = action;

    String response;
    serializeJson(res, response);
    client.publish(topic_response, response.c_str());
}

if (String(topic) == topic_sync) {
    if (doc.containsKey("devices")) {
        for (JsonObject dev : doc["devices"].as<JsonArray>()) {
            int id = dev["DeviceID"];
            String action = dev["Action"];
            applyState(id, action);
        }
    }
}

// RECONNECT
void reconnect() {
    while (!client.connected()) {
        if (client.connect("ESP32_FULLNODE", mqtt_user, mqtt_pass)) {
            Serial.println("MQTT Connected");

            client.subscribe(topic_control);
            client.subscribe(topic_sync);

            requestSync();
        } else {
            delay(2000);
        }
    }
}

void setup() {
    Serial.begin(115200);

    pinMode(LED1, OUTPUT);
    pinMode(LED2, OUTPUT);
}

```

```

pinMode(LED3, OUTPUT);

digitalWrite(LED1, LOW);
digitalWrite(LED2, LOW);
digitalWrite(LED3, LOW);

Wire.begin(SDA_PIN, SCL_PIN);
dht.begin();
lightMeter.begin();

setup_wifi();

client.setServer(mqtt_server, mqtt_port);
client.setCallback(callback);
}

void loop() {

    if (!client.connected()) {
        reconnect();
    }

    client.loop();

    unsigned long now = millis();

    if (now - lastSend >= interval) {
        lastSend = now;

        float temp = dht.readTemperature();
        float hum  = dht.readHumidity();
        float lux  = lightMeter.readLightLevel();

        if (isnan(temp) || isnan(hum)) return;

        bool newState = led1State;
        if (lux < 10) {
            newState = false;
        } else {
            newState = true;
        }

        if (newState != led1State) {

```

```

    led1State = newState;

    digitalWrite(LED1, led1State ? HIGH : LOW);

    StaticJsonDocument<100> res;
    res["DeviceID"] = 1;
    res["Action"] = led1State ? "ON" : "OFF";
    res["Type"] = "AUTO";

    String response;
    serializeJson(res, response);
    client.publish(topic_response, response.c_str());
}

StaticJsonDocument<200> doc;
doc["temperature"] = temp;
doc["humidity"] = hum;
doc["light"] = lux;

String payload;
serializeJson(doc, payload);

client.publish(topic_sensor, payload.c_str());
}
}

```

- Cơ chế hoạt động của phần cứng:
 - Hệ thống phần cứng hoạt động theo hai hướng chính: đọc dữ liệu môi trường và điều khiển thiết bị.
 - Trước hết, ESP32 khởi tạo kết nối WiFi và kết nối đến MQTT broker. Sau khi kết nối thành công, thiết bị sẽ tiến hành đọc dữ liệu từ DHT11 và BH1750 theo chu kỳ 2 giây. Giá trị nhiệt độ, độ ẩm và ánh sáng được đóng gói thành gói JSON rồi gửi lên topic DataSensors.
 - Ở chiều ngược lại, khi server hoặc frontend gửi lệnh điều khiển xuống topic DeviceControl, ESP32 sẽ nhận dữ liệu qua hàm callback, phân tích JSON để lấy DeviceID và Action. Sau đó, ESP32 sẽ gọi hàm applyState() để bật hoặc tắt LED tương ứng. Ngoài ra, hệ thống còn có cơ chế đồng bộ trạng thái ban đầu qua topic DeviceSyncRequest và DeviceSync, giúp thiết bị cập nhật trạng thái đồng nhất khi khởi động lại hoặc mất kết nối.

2. Backend

2.1. Tổng quan

Backend của hệ thống IoT được xây dựng bằng Node.js kết hợp với Express, đóng vai trò là lớp trung gian xử lý toàn bộ dữ liệu giữa thiết bị phần cứng, cơ sở dữ liệu và giao diện người dùng. Backend có nhiệm vụ nhận dữ liệu cảm biến từ thiết bị ESP32 thông qua giao thức MQTT, lưu trữ dữ liệu vào MySQL, cung cấp các API cho frontend và gửi lệnh điều khiển ngược trở lại thiết bị. Bên cạnh đó, hệ thống còn tích hợp Socket.IO để đẩy dữ liệu thời gian thực đến giao diện web, giúp người dùng theo dõi trạng thái hệ thống ngay khi dữ liệu thay đổi

2.2. Công nghệ sử dụng

Backend của dự án sử dụng một số công nghệ chính sau:

- Node.js là môi trường chạy JavaScript phía server, cho phép xây dựng các ứng dụng web có khả năng xử lý bất đồng bộ tốt, phù hợp với hệ thống IoT cần nhận và gửi dữ liệu liên tục.
- Express là framework chạy trên Node.js, được dùng để xây dựng các route, xử lý request/response và tổ chức API theo cấu trúc rõ ràng.
- MySQL là hệ quản trị cơ sở dữ liệu quan hệ dùng để lưu trữ dữ liệu cảm biến, lịch sử điều khiển thiết bị và trạng thái hoạt động của hệ thống.
- MQTT là giao thức truyền thông nhẹ, được dùng để trao đổi dữ liệu giữa ESP32 và backend theo mô hình publish/subscribe.
- Socket.IO được sử dụng để cập nhật dữ liệu thời gian thực từ server đến frontend mà không cần tải lại trang.

2.3. Cấu trúc thư mục

src/index.js: file khởi chạy chính của server.

src/config/db.js: cấu hình kết nối cơ sở dữ liệu MySQL.

src/config/mqtt.js: cấu hình kết nối và xử lý dữ liệu MQTT.

src/controllers/: chứa các controller xử lý logic nghiệp vụ.

src/routes/: chứa khai báo các tuyến API.

src/socket.js: cấu hình Socket.IO.

src/swagger.js: cấu hình tài liệu API bằng Swagger.

2.4. Tích hợp MQTT

Backend kết nối tới MQTT broker bằng thông tin cấu hình từ biến môi trường như MQTT_BROKER, USER_NAME, PASSWORD. Sau khi kết nối thành công, backend subscribe ba topic chính

- MQTT_TOPIC: nhận dữ liệu cảm biến.

- TOPIC_RESPONSE: nhận phản hồi từ thiết bị sau khi thực hiện lệnh điều khiển.
- TOPIC_SYNC_REQUEST: nhận yêu cầu đồng bộ trạng thái từ thiết bị.

Khi nhận dữ liệu từ topic cảm biến, backend phân tích JSON payload, lấy ra các giá trị temperature, humidity, light rồi lưu đồng thời vào MySQL. Sau đó, dữ liệu cũng được phát qua Socket.IO để frontend cập nhật ngay lập tức.

Khi nhận phản hồi từ topic DeviceResponse, backend sẽ kiểm tra lịch sử điều khiển đang ở trạng thái Processing, sau đó cập nhật thành Success trong database và gửi trạng thái mới lên frontend.

Khi nhận topic yêu cầu đồng bộ DeviceSyncRequest, backend sẽ truy vấn trạng thái gần nhất của từng thiết bị trong bảng actionshistory, rồi publish lại danh sách trạng thái này xuống topic DeviceSync để ESP32 cập nhật trạng thái đầu vào cho đồng bộ hệ thống.

3. FrontEnd

3.1. Tổng quan

Frontend của hệ thống được xây dựng nhằm cung cấp giao diện trực quan giúp người dùng theo dõi dữ liệu cảm biến và điều khiển thiết bị từ xa. Hệ thống frontend sử dụng các công nghệ web cơ bản gồm HTML, CSS và JavaScript, giúp đảm bảo tính đơn giản, dễ triển khai và phù hợp với các ứng dụng IoT quy mô nhỏ và trung bình.

Frontend đóng vai trò là lớp hiển thị, kết nối với backend thông qua các API và nhận dữ liệu thời gian thực thông qua Socket.IO. Nhờ đó, người dùng có thể quan sát dữ liệu cảm biến như nhiệt độ, độ ẩm, ánh sáng cũng như trạng thái thiết bị mà không cần tải lại trang.

3.2. Công nghệ sử dụng

Frontend của hệ thống sử dụng các công nghệ chính sau:

- HTML: xây dựng cấu trúc giao diện, bố trí các thành phần như bảng dữ liệu, biểu đồ, nút điều khiển.
- CSS: định dạng giao diện, giúp hệ thống có bố cục rõ ràng, dễ nhìn và thân thiện với người dùng.
- JavaScript: xử lý logic phía client, gọi API từ backend và cập nhật dữ liệu lên giao diện.
- Socket.IO Client: nhận dữ liệu realtime từ server, giúp dashboard cập nhật ngay khi có thay đổi.

Socket.IO là thư viện hỗ trợ giao tiếp hai chiều theo thời gian thực giữa client và server, giúp frontend nhận dữ liệu ngay khi server gửi mà không cần polling liên

tục. Điều này đặc biệt phù hợp với các hệ thống dashboard IoT cần cập nhật dữ liệu liên tục.

3.3. Cấu trúc thư mục

FrontEnd/

├── dashboard.css :

├── dashboard.html :

├── dashboard.js :

├── history.css

├── history.html

├── history.js

├── profile.css

├── profile.html

├── sensors.css

├── sensors.html

└── sensors.js

- Ý nghĩa từng file:
 - + dashboard.css: chịu trách nhiệm định dạng giao diện cho trang Dashboard
 - + dashboard.html: định dạng giao diện cho trang dữ liệu cảm biến đóng vai trò là màn hình tổng quan (Dashboard)
 - + dashboard.js: Đây là file xử lý logic chính của trang Dashboard. File này có nhiệm vụ:
 - Gọi API để lấy dữ liệu cảm biến ban đầu.
 - Tạo và cập nhật biểu đồ bằng Chart.js.
 - Kết nối Socket.IO để nhận dữ liệu realtime.
 - Gửi lệnh bật/tắt thiết bị lên backend.
 - Cập nhật trạng thái công tắc theo phản hồi từ server.
 - + sensors.html: Trang này dùng để hiển thị dữ liệu cảm biến theo dạng bảng. Người dùng có thể xem danh sách các bản ghi đã lưu, lọc theo loại cảm biến như nhiệt độ, độ ẩm, ánh sáng, và tìm kiếm theo thời gian.
 - + sensors.css: Định dạng giao diện cho trang dữ liệu cảm biến

- + sensors.js: File xử lý logic cho trang dữ liệu cảm biến. Chức năng chính gồm:
 - Gọi API lấy danh sách cảm biến từ backend.
 - Hỗ trợ tìm kiếm theo ngày giờ.
 - Lọc dữ liệu theo loại cảm biến.
 - Hiển thị dữ liệu vào bảng.
 - Tạo phân trang để người dùng dễ duyệt nhiều bản ghi.
- + history.html: Trang này dùng để hiển thị lịch sử thao tác điều khiển thiết bị. Người dùng có thể xem thiết bị nào đã được bật/tắt, hành động nào đã được gửi đi, trạng thái thực hiện lệnh và thời gian thực hiện.
- + history.css: File CSS này quy định giao diện của trang lịch sử điều khiển. Nó định dạng bảng dữ liệu, thanh tìm kiếm, bộ lọc thiết bị và khu vực phân trang.
- + history.js: file xử lý logic cho trang lịch sử điều khiển thiết bị. Nó có các chức năng:
 - Gọi API lấy dữ liệu lịch sử từ backend.
 - Tìm kiếm theo ngày giờ.
 - Lọc theo thiết bị như đèn, quạt, điều hòa.
 - Phân trang dữ liệu.
 - Hiển thị trạng thái lệnh như Thành công, Đang xử lý, Thất bại.
- + profile.html: hiển thị thông tin cá nhân của người thực hiện dự án
- + profile.css: định dạng giao diện cho trang Profile

4. Tài liệu API

4.1. Endpoint lấy dữ liệu cảm biến

GET /api/sensors/data – Lấy dữ liệu cảm biến từ hệ thống (phục vụ dashboard hoặc lịch sử).

Request:

- Method: GET
- Headers:
 - + Content-Type: application/json

Query Parameters (tùy chọn):

- range: Khoảng thời gian (ví dụ: 10days)
- search: Tìm kiếm theo giá trị hoặc thời gian
- sensor: Lọc theo loại cảm biến
- limit: Số bản ghi mỗi trang
- page: Trang hiện tại

Response:

```
{
  "data": [
    {
      "id": 1,
```



```

    "temperature": 30.2,
    "humidity": 66,
    "light": 30.5,
    "timestamp": "2025-09-29T08:51:40.214Z"
  }
],
"total": 100,
"page": 1,
"totalPages": 10
}

```

4.2. Endpoint lấy lịch sử điều khiển thiết bị

GET /api/devices/data – Lấy lịch sử điều khiển thiết bị.

Request:

- Method: GET
- Headers:
- + Content-Type: application/json

Query Parameters (tùy chọn):

- search: Tìm theo thời gian
- deviceId: Lọc theo thiết bị
- status: Lọc theo trạng thái
- limit: Số bản ghi mỗi trang
- page: Trang hiện tại

Response:

```

{
  "data": [
    {
      "id": 10,
      "deviceName": "LED 1",
      "action": "ON",
      "status": "Success",
      "createdAt": "2025-09-29T08:55:00.000Z"
    }
  ],
  "total": 50,
  "page": 1,
  "totalPages": 5
}

```

4.3. Endpoint lấy trạng thái thiết bị

GET /api/devices/status – Lấy trạng thái hiện tại của các thiết bị.

Request:

- Method: GET
- Headers:
- + Content-Type: application/json

Response:

```
{
  "devices": [
    {
      "deviceId": 1,
      "status": "ON"
    },
    {
      "deviceId": 2,
      "status": "OFF"
    },
    {
      "deviceId": 3,
      "status": "ON"
    }
  ]
}
```

4.4. Endpoint điều khiển thiết bị

POST /api/devices/control – Gửi lệnh điều khiển thiết bị.

Request:

- Method: POST
- Headers:
- + Content-Type: application/json
- Body:

```
{
  "DeviceID": 1,
  "Action": "ON"
}
```

Response:

```
{
  "message": "Command sent successfully",
  "status": "Processing"
}
```

Chương 4: Kết luận và đánh giá

4.1. Kết quả đạt được

Sau quá trình xây dựng, dự án đã hoàn thiện một hệ thống IoT có khả năng kết nối giữa phần cứng, máy chủ và giao diện web, đáp ứng được mục tiêu giám sát và điều khiển thiết bị từ xa. Ở phía phần cứng, hệ thống sử dụng ESP32 làm vi điều khiển trung tâm, kết hợp với cảm biến DHT11 để đo nhiệt độ, độ ẩm và cảm biến BH1750 để đo cường độ ánh sáng. Dữ liệu từ các cảm biến được ESP32 thu thập định kỳ, sau đó đóng gói theo định dạng JSON và gửi lên MQTT broker. Đồng thời, hệ thống còn sử dụng các chân GPIO để điều khiển ba thiết bị đầu ra mô phỏng bằng LED, giúp kiểm chứng khả năng nhận lệnh và phản hồi của phần cứng.

Ở phía backend, dự án đã xây dựng được một server bằng Node.js và Express, có nhiệm vụ tiếp nhận dữ liệu từ MQTT, xử lý logic nghiệp vụ, lưu trữ dữ liệu vào MySQL và cung cấp API cho frontend. Backend còn có khả năng gửi lệnh điều khiển từ người dùng xuống thiết bị thông qua MQTT, đồng thời quản lý trạng thái lệnh điều khiển theo các trạng thái như Processing, Success và Fail. Ngoài ra, việc tích hợp Socket.IO giúp hệ thống có thể cập nhật dữ liệu realtime lên giao diện người dùng mà không cần tải lại trang, làm tăng tính trực quan và trải nghiệm sử dụng.

Về phía frontend, dự án đã xây dựng được các trang giao diện chính gồm Dashboard, Sensors, History và Profile. Trang Dashboard cho phép hiển thị dữ liệu cảm biến theo thời gian thực, đồng thời cung cấp khu vực điều khiển thiết bị. Trang Sensors hỗ trợ xem danh sách dữ liệu cảm biến, tìm kiếm và lọc theo loại cảm biến. Trang History cho phép theo dõi lịch sử điều khiển thiết bị, còn trang Profile cung cấp thông tin cá nhân và tài nguyên liên quan đến dự án. Nhìn chung, frontend đã đáp ứng được yêu cầu hiển thị dữ liệu, điều khiển hệ thống và tra cứu lịch sử hoạt động một cách rõ ràng, trực quan.

Từ tổng thể, dự án đã xây dựng thành công một mô hình IoT hoàn chỉnh với luồng hoạt động đầy đủ: cảm biến → ESP32 → MQTT → Backend → MySQL → Frontend, đồng thời hỗ trợ luồng điều khiển ngược từ người dùng về lại thiết bị. Hệ thống đã thể hiện được tính đồng bộ giữa phần cứng và phần mềm, phù hợp với một bài toán IoT thực tế ở mức cơ bản đến trung bình.

4.2. Đánh giá ưu điểm

Một trong những ưu điểm lớn nhất của dự án là hệ thống được xây dựng theo kiến trúc khá rõ ràng và tách biệt chức năng giữa các lớp. Phần hardware chịu trách nhiệm thu thập dữ liệu và điều khiển thiết bị, backend xử lý logic và truyền thông, còn frontend đảm nhiệm hiển thị và tương tác với người dùng. Cách tổ chức này giúp hệ thống dễ hiểu, dễ kiểm thử và dễ mở rộng trong tương lai.

Tiếp theo là dự án đã sử dụng đúng các công nghệ phù hợp với mô hình IoT. ESP32 là một vi điều khiển mạnh, có tích hợp WiFi nên rất phù hợp để gửi dữ liệu lên mạng. MQTT là giao thức nhẹ, tối ưu cho thiết bị nhúng và truyền dữ liệu thời gian thực. Node.js và Express giúp backend xử lý bất đồng bộ tốt, phù hợp với việc nhận dữ liệu liên tục từ thiết bị. MySQL giúp lưu trữ dữ liệu ổn định và có cấu trúc rõ ràng, thuận tiện cho truy vấn lịch sử. Việc kết hợp các công nghệ này giúp hệ thống hoạt động tương đối mượt và đúng với mô hình IoT hiện đại.

Một điểm mạnh nữa là hệ thống có hỗ trợ realtime. Nhờ MQTT và Socket.IO, dữ liệu cảm biến và trạng thái thiết bị có thể được cập nhật gần như ngay lập tức trên giao diện web. Điều này làm cho người dùng có thể quan sát sự thay đổi của hệ thống theo thời gian thực, thay vì phải làm mới trang thủ công. Đây là một tính năng rất quan trọng trong các ứng dụng giám sát và điều khiển từ xa.

4.3. Đánh giá về nhược điểm

Về cảm biến, hệ thống mới chỉ sử dụng DHT11 và BH1750 nên dữ liệu giám sát còn khá cơ bản. DHT11 có độ chính xác ở mức vừa phải, phù hợp cho mô hình thử nghiệm nhưng chưa thật sự tối ưu nếu dùng trong môi trường yêu cầu độ chính xác cao.

Ở phía backend, hệ thống tuy đã có API và realtime nhưng vẫn còn phụ thuộc khá nhiều vào luồng xử lý thủ công trong từng module. Nếu số lượng thiết bị hoặc lượng dữ liệu tăng lớn, backend cần tối ưu thêm về hiệu năng, chia nhỏ service hoặc bổ sung hàng đợi xử lý để đảm bảo tính ổn định lâu dài. Ngoài ra, hệ thống hiện chưa thấy phân xác thực người dùng, phân quyền hay bảo vệ truy cập API, nên mức độ an toàn chưa cao nếu triển khai trên môi trường thật.

Frontend của dự án đã đáp ứng tốt chức năng cơ bản, nhưng giao diện vẫn còn thiên về mô hình truyền thống, chưa sử dụng framework hiện đại như React hay Vue. Điều này giúp code đơn giản và dễ hiểu, nhưng về lâu dài sẽ hạn chế khi cần xây dựng giao diện phức tạp, quản lý trạng thái lớn hoặc tái sử dụng component. Bên cạnh đó, mức độ tối ưu cho thiết bị di động và khả năng responsive có thể chưa hoàn thiện tuyệt đối.

Một hạn chế khác là hệ thống chủ yếu hoạt động trong môi trường mạng nội bộ hoặc mạng có cấu hình cố định. Nếu muốn triển khai từ xa qua Internet thật sự, cần thêm các giải pháp bảo mật, NAT traversal, domain, SSL/TLS hoặc cloud broker ổn định hơn. Điều này cho thấy hệ thống hiện phù hợp hơn với mục đích học tập, demo và nghiên cứu.

4.4. Kết luận

Nhìn chung, dự án đã đạt được mục tiêu xây dựng một hệ thống IoT có đầy đủ các thành phần quan trọng gồm phần cứng, backend, cơ sở dữ liệu và frontend. Hệ

thống đã chứng minh được khả năng thu thập dữ liệu cảm biến, lưu trữ, hiển thị realtime và điều khiển thiết bị từ xa. Đây là một nền tảng tốt để tiếp tục phát triển thành một hệ thống hoàn chỉnh hơn trong tương lai. Dù vẫn còn một số hạn chế về phân cứng, bảo mật và khả năng mở rộng, nhưng xét trên phạm vi đồ án, dự án đã đạt kết quả khá tốt và thể hiện rõ kiến thức về IoT, lập trình backend, frontend và giao tiếp thiết bị.